Techworks Best Practices in AI

version

Best Practices Working Group

November 24, 2023

Contents

Welcome to the Techworks Best Practices in AI Guide 1
- Executive Summary 1
- Using this Guide 1
  - Application Specific Requirements 2
    - AI Risk 2
    - Data Protection 2
    - Safety, Security and Robustness 2
    - Transparency and Explainability 3
    - Fairness 3
    - Accountability and Governance 3
    - Contestability and Redress 3
- Should I Use AI? 4
- Defining the Project 4
- Collecting Data 4
- Training Your AI Application 5
- Deploying your AI Application 5
- Task List 6
  - Should I Use AI? 6
    - Evaluate Engineering Case 6
      - Examples 6
    - Assess Unacceptable AI Risk 7
      - Examples 7
    - Assess Problematic Data Risk 7
      - Examples 8
  - Defining the Project 8
    - Define Goals and Metrics 8
    - Define Limitations and Boundaries 8
    - Complete a Project Risk Assessment 8
  - Collecting Data 8
    - Creating and Collecting your Data Set 8
      - Defining the Plan 8
      - Implementing the Plan 9
      - Examples 10
    - Version Control, CI/CD for Data 11
      - Examples 11
    - Documentation 11
      - Examples 12
    - Logging 12
    - Data Exploration 12
    - Cleaning your Data 15
    - Validation and Testing 15
    - Scaling and Automation: Data Collection 15
    - Scaling and Automation: Data Storage 15
    - Scaling and Automation: Data Access 15

Training Your AI Application 15
Choosing Your AI approach 15
Version Control, CI/CD, Training 15
Documentation and Logging, Training 15
Data Pre-processing 15
Training Infrastructure 15
Creating a Training Process 15
Testing and Validation, Training 15
Exploring Outcomes and Biases 15
Scaling and Automation: Training Pipeline 15
Deploying your AI Application 15
Version Control, CI/CD, Deployment 15
Documentation and Logging, Deployment 15
Preparing a Trained Model 15
Deployment Infrastructure 15
Deploying Your Model 15
Testing and Validation, Deployment 15
Model Monitoring 15
Scaling and Automation: Deployment Pipeline 15
Examples 16
Example Engineering Document 16
Example Dataset Creation Document 16
Contributors 16
Authors and Contributors 16
Reviewers 16
Indices and tables 16

Welcome to the Techworks Best Practices in AI Guide

This document is the best practices in AI guide for electronic systems engineers produced by the Best Practices in AI workstream at TechNES. This guide is currently maintained in Sphinx on GitHub.

While we maintain our core document in Sphinx, we're aware that this may not fit everybody's day-to-day workflows. We therefore currently publish versions of this document in the following formats:

- Online
- PDF
- docx

We welcome any and all contributors to this guide. You don't necessarily need to author content to contribute - feeding back on the guide is a valuable contribution. We current coordinate group activities via Basecamp. If you want to get involved then:

- Get in touch

We're also very happy to take feedback on this guide directly. You can do this by:

- Sending us your comments
- Opening an issue or a pull request on Github.

Executive Summary

Artificial Intelligence (AI) and Machine Learning (ML) technologies are key to many modern engineering projects due to their ability to solve many problems that are difficult or impossible with other methods. While most engineers will find themselves enjoying a significant overlap between these techniques and their existing skill set, they are also liable to find that AI and ML is its own field with its own unique demands and (often hidden) pitfalls. While there are many resources available for self teaching, it is generally assumed the practitioner is either an absolute beginner to engineering, or already a seasoned expert in AI and ML. In this document, we provide a practical guide to AI and ML for electronic systems engineers who already have a strong base of knowledge in electronic systems but no specialized expertise. This guide will be practice focused, with the goal of helping engineers to make good decisions and avoid problems.

The guide will cover, in order:

- Should I Use AI?
- Defining the Project
- Collecting Data
- Training your AI Application
- Deploying your AI Application

Using this Guide

This guide is laid out as a series of steps taking you from conceptualizing your AI application through to deploying it in the real world. These steps are broken into sections, and set out in the order they should be initially considered. It's strongly suggested you don't "skip ahead" to later steps from earlier ones. However, it is expected reviewing later sections may prompt you to reevaluate earlier steps and return to previous decisions.

Each step discusses a problem to be solved, what needs to be done to solve it, and what evidence needs to be provided to support this. The steps are enumerated in a table available in each section, with details on each entry available further down the document. The table is to be filled in as the steps are worked through and lists 3 columns. The first lists the name of the requirement. The second is for recording the evidence supporting the fulfillment of the requirement, and the third is a checkbox to note when the requirement has been completed. An example table with the first requirement filled out is below:

Requirement	Evidence	Complete
Example requirement 1	Example Risk Assessment.pdf, Example document.pdf	Yes

Example requirement 2	•	•
…	•	•

Not every part of every step will need to be fulfilled for every application. Different applications may require different approaches. We discuss some recurring issues that many engineers will have to deal with these below.

Application Specific Requirements

AI Risk

Direct regulation of AI in the EU is tending to a risk based approach based on the harm than an AI application poses to people. Applications are classified into risk categories, and those that pose a high risk are to be subject to more stringent requirements. These categories are currently:

- Unacceptable risk. Applications that are a clear threat to the safety, livelihoods or rights of people.
- High risk. Applications that are critical to the safety or wellbeing of people, that require special mitigations to adequately cover the risks they pose
- Limited risk. Applications with lower impacts to safety or wellbeing, but which still require some level of mitigation
- Minimal or no risk. Applications with minimal to zero risk.

Providing guidance on any upcoming or current legislation is out of the scope of this guide. However, this categorization of the risk that AI applications pose is a useful tool for discussing best engineering practices. These risk levels will recur in this guide, and we will refer to them as AI Risk.

Data Protection

AI models are data driven. This leaves them significantly subject to data protection regulation, especially GDPR. According to GDPR, if you process data, you have to do so according to seven protection and accountability principles:

- Lawfulness, fairness and transparency — Processing must be lawful, fair, and transparent to the data subject.
- Purpose limitation — You must process data for the legitimate purposes specified explicitly to the data subject when you collected it.
- Data minimization — You should collect and process only as much data as absolutely necessary for the purposes specified.
- Accuracy — You must keep personal data accurate and up to date.
- Storage limitation — You may only store personally identifying data for as long as necessary for the specified purpose.
- Integrity and confidentiality — Processing must be done in such a way as to ensure appropriate security, integrity, and confidentiality (e.g. by using encryption).
- Accountability — The data controller is responsible for being able to demonstrate GDPR compliance with all of these principles.

There are also certain categories of data that must be given special consideration. Providing guidance on legislation remains out of the scope of this guide. However, once again, the ideas put forward in GDPR are important in framing the discussion of best engineering practices, and will recur in this guide.

Safety, Security and Robustness

A core requirement of any system is that it functions correctly, both in conditions of normal use, as well as conditions of misuse (intentional or otherwise), or other adverse conditions. An AI system should be:

- Safe: either functioning correctly or failing safely in all conditions
- Secure: resilient against misuse or abuse (intentionally or otherwise), and

- Robust: ensuring functionality in as wide a range of conditions as possible

While the need to meet these challenges is not unique to AI systems, AI systems bring several unique problems to each of these areas. For example:

- Safety is a continual challenge in AI systems that are fundamentally statistical. Guaranteeing safe behavior in the wide range of situations an automotive AI may encounter, for example, remains a continuous challenge.
- AI models present unique attack surfaces that must be secured. For example, in many situations, models will leak information about the data that they were trained on.
- Similarly, AI models struggle to robustly deal with novel data outside of their training set. This can both cause problems in normal use, and be exploited by attackers.

Safety, security and robustness will be important ideas throughout this document.

Transparency and Explainability

It is important in many cases that we can understand how our systems function. This imperative should be familiar to any electronic systems engineer, through the value of clear code and documentation. We use two terms to describe these requirements:

- Transparency: the communication of appropriate information about an AI system to relevant people (for example, information on how, when, and for which purposes an AI system is being used), and
- Explainability: the extent to which it is possible for relevant parties to access, interpret and understand the decision-making processes of an AI system

Many AI systems are uniquely troublesome in these respects. Many common methods, such as Neural Networks are effectively “black boxes”. These methods provide a solution, but through a means that is ultimately not human interpretable.

Fairness

Fairness is another core requirement of any system, especially in light of the above ideas of transparency and explainability. By fairness, we mean a system that:

- Does not undermine the legal rights of individuals or organizations, discriminate unfairly against individuals or create unfair market outcomes.

Fairness is a challenge in AI systems that learn from data. The decisions these systems make are a reflection of the patterns in the data they are trained on. If the data is biased, the systems trained on them will also be biased. This has been proven expensively and at length in the real world, for example through attempts at creating criminal justice AI or hiring AI.

Accountability and Governance

Any system that has the potential to cause harm requires oversight. Any AI system will therefore require systems of:

- Governance, a framework for managing the development, deployment and operation of AI
- Accountability, clear lines of responsibility for all aspects of the AI applications

The full scope of requirements is unlikely to be fully addressed by engineering. Nonetheless, these principles are important in the greater context of the AI application, and will be an important part of development.

Contestability and Redress

Any system that has the potential to cause harm to people also requires ways for this harm to be recognised and reversed. Any such AI system will therefore require systems of:

- Contestability, those who may be adversely affected must have a route to contest decisions or outcomes
- Redress, there must be a system in place to assess these concerns, and redress the affected party when necessary

The full scope of requirements is unlikely to be fully addressed by engineering. Nonetheless, these principles are important in the greater context of the AI application, and will be an important part of development.

Should I Use AI?

Developing an AI application can present significant challenges. Collection of data, testing and validation are challenges. As seen in the previous section, many applications of AI will come with special requirements that can be a challenge in themselves. To address this, the very first step in this guide is to be able answer the following: should I use AI to solve my problem? We break this problem down into two parts:

- What is the engineering case for using AI to solve the problem, over other approaches?
- Does the problem touch on any application or data areas that may effectively prohibit development?
    - What is the risk that the project falls into the unacceptable AI Risk category?
    - What is the risk that the project makes use of problematic data?

Requirement	Evidence	Complete
Evaluate Engineering Case	•	
Assess Unacceptable AI Risk	•	
Assess Problematic Data Risk	•	

Defining the Project

In the previous step we worked on an engineering case for AI in our project. In this step, we will take the first step in realizing our project by setting the goals and bounds of the project. There are three steps to complete in this section:

- Defining goals and metrics for success
- Defining limitations and boundaries on the project
- Completing a risk assessment of the project as a whole

Requirement	Evidence	Complete
Define Goals and Metrics	•	
Define Limitations and Boundaries	•	
Complete a Project Risk Assessment	•	

Collecting Data

In the previous step, we defined the scope of our project. In this step, we move on to the first part of the practical engineering of our AI project, collecting the data. It is a core requirement of any AI application that it is data driven. This means that some data will need to be collected, or at the very least, processed. It's very important to get this right, as the strength of the data will have a strong effect on the efficacy of training and deploying our AI application. We set out a number of steps for this section, but our primary challenges are:

- Making sure the data we're is collecting useful, truthful, and effective data
- Making sure we transform our raw data into a form that can effectively utilized by AI algorithms
- Making sure our infrastructure for collection, storage, and access is appropriate and robust

Requirement	Evidence	Complete
Creating and Collecting your Data Set	•	
Version Control, CI/CD for Data	•	
Documentation	•	
Logging	•	

Data Exploration	•	
Cleaning your Data	•	
Validation and Testing	•	
Scaling and Automation: Data Collection	•	
Scaling and Automation: Data Storage	•	
Scaling and Automation: Data Access	•	

Training Your AI Application

In the previous step, we collected the data for our AI project. In this step, we will make use of it by using it to train an AI algorithm of our choice to meet the goals of our project. This is also the step where systematic problems from decisions in earlier steps are likely to start manifesting in force. We strongly suggest that readers don't hesitate to revisit earlier decisions at this stage if they prove to be unfruitful. Once again, we set out a number of steps for this section, but our primary challenges are:

- Establishing which AI approach we're going to use
- Engineering a pipeline to train our approach in the best possible way
- Building confidence that this training results in an AI algorithm that does all the things it should, and none of the things it shouldn't

Requirement	Evidence	Complete
Choosing Your AI approach	•	
Version Control, CI/CD, Training	•	
Documentation and Logging, Training	•	
Data Pre-processing	•	
Training Infrastructure	•	
Creating a Training Process	•	
Testing and Validation, Training	•	
Exploring Outcomes and Biases	•	
Scaling and Automation: Training Pipeline	•	

Deploying your AI Application

After training our AI application, we can finally deploy it and (hopefully) achieve the goals set out in our previous steps. This step will likely represent a welcome return to familiarity for the professional engineer, as the process for deploying an AI application is fairly similar to that of deploying any other software application. Our process will proceed broadly in three steps:

- Preparing our trained the model for a live environment
- Engineering a process for deployment and model updating
- Setting up continuous monitoring for our model

Requirement	Evidence	Complete

Version Control, CI/CD, Deployment	•	
Documentation and Logging, Deployment	•	
Preparing a Trained Model	•	
Deployment Infrastructure	•	
Deploying Your Model	•	
Testing and Validation, Deployment	•	
Model Monitoring	•	
Scaling and Automation: Deployment Pipeline	•	

Task List

Should I Use AI?

Evaluate Engineering Case

As we discussed previously, while AI is a powerful tool in many applications, it may not be immediately appropriate for the problem you are trying to solve. This step requires you to enumerate and assess what available alternative approaches to solving the problem, and how they compare to an AI solution. This isn't intended as a deep dive into the details and risks of your prospective systems, but a high level check that this engineering approach is right for you.

This step is fairly short, and to complete it, you simply need to list the viable approaches to solving your problem (including the AI approach), and compare them. This should include solutions that might not take a data driven approach, as well as data driven approaches that don't use learning, for example, heuristic based approaches. The choice of how to compare these will be dictated by your use case, but you're likely to want to consider the following:

- What are the financial burdens of each case?
- What are the engineering burdens of each case?
- What are the legal burdens?

Examples

In this example, we work through the engineering case for a phishing email detector. There is an example engineering case document, with a discussion of the problem below.

Our use case is a filter to reduce the success rate of phishing emails. As an example of how we might think about this, we consider three alternative internal solutions:

- A non data driven (or minimally data driven) approach. All emails that are from external email addresses automatically have a header attached warning employees that the sender may not be trustworthy, and to apply caution in respect of the contents.
- A data driven approach that uses engineer designed heuristics instead of a learning approach. For example, emails are filtered if they:
    - Are from a list of known "bad" addresses
    - Contain too many known "bad" words, e.g. "low interest rate loans" Has ".exe" attachments
- An AI approach that learns from a corpus of previous emails to read an email's content, and classifies it as phishing, or not phishing.

The respective advantages of each approach are:

- The non data driven approach is simple to implement, requires few resources to execute, and is simple and cheap to maintain. It will never incorrectly filter out legitimate emails, as it passes the burden of decision making to employees (with extra information).
- The heuristic approach is also fairly simple to implement, and requires few resources to execute. It is also very easy to tune for specific requirements (e.g. never do this, always do this).
- The AI approach will likely have the best performance at identifying spam emails, if trained on sufficient data. It is also likely to adapt fairly well to changes in phishing approaches over time, if continually fed with data.

The disadvantages of each are:

- The non-data driven approach provides extra information, but the decision making ultimately places responsibility on the end user.
- The heuristic approach requires each rule to be created manually, scaling poorly to large scale problems. Creating effective heuristic rules that do not result in any real emails being filtered out is also likely to be challenging. Adapting to new phishing approaches is also likely to be a significant burden, requiring runes to be written or rewritten.
- The AI approach will need a large corpus of data to learn from which will have to be prepared appropriately. It will flag real emails incorrectly some of the time (no matter how well trained), and this may not be easy to fix. Adoption will require more data preparation and training.

Picking the best approach (or approaches) to use depends on our context. In this particular example, our non-AI examples are significantly simpler to implement, and are likely to do better for small scale problems. Other conditions may also influence the decision. An organization of phishing-aware people that mostly communicate internally may hugely benefit from the first approach.

Assess Unacceptable AI Risk

In the Using This Guide section, we defined AI risk in several categories: Unacceptable, High, Limited, and Minimal. Before continuing any further in this guide, it is important to consider the risk that your project may fall into the Unacceptable category. These are applications that are a clear threat to the safety, livelihoods or rights of people, and are not suitable to be tackled with AI. Examples include:

- AI systems using subliminal techniques, or manipulative or deceptive techniques to distort behavior
- AI systems exploiting vulnerabilities of individuals or specific groups
- Biometric categorization systems based on sensitive attributes or characteristics
- AI systems used for social scoring or evaluating trustworthiness
- AI systems used for risk assessments predicting criminal or administrative offenses
- AI systems creating or expanding facial recognition databases through untargeted scraping
- AI systems inferring emotions in law enforcement, border management, the workplace, and education

At this stage, you may not have fully fleshed out the scope of your application. Nonetheless, this initial assessment is important to pre-empt this risk. If you believe your application may fall into this category, discontinue further work on it until you have resolved this issue.

To complete this step, complete a risk assessment on the harms your AI project may pose to the safety, livelihoods or rights of people.

Examples

Assess Problematic Data Risk

Some types of data are subject to extra difficulties that will either require extra licensing, oversight, or may be effectively impracticable to collect. This section is about evaluating the risk that the data you are likely to wish to collect is available within the constraints of your business. Note that this section isn't a dive into data collection requirements under GDPR, but a higher level feasibility check. Some examples of types of data that will be problematic:

- Criminal Conviction Data, only processable:

    - under the control of official authority; or
    - authorized by domestic law.
- Data collected by experimenting on humans or animals
    - requiring extra licensing and oversight
- Data surrounding experimentation with infectious diseases

The specifics of problematic data will depend on the domain the data is being collected in. At this stage, you may not have fully fleshed out the scope of your application. Nonetheless, you should, before proceeding further, assess the risk that this applies to you. If you believe your application may fall into this category, you should once again discontinue further work on it until you have resolved this issue.

To complete this step, complete a risk assessment on potential problems surrounding the collection of the type of data you are likely to require.

Examples

Defining the Project

Define Goals and Metrics

Define Limitations and Boundaries

Complete a Project Risk Assessment

Collecting Data

Creating and Collecting your Data Set

Defining the Plan

The first step in creating our AI application is to create and (with caveats) implement a plan to collect a dataset to drive your AI application. The plan will include:

- What data you are going to collect
- Where/whom you are going to collect it from
- How you are going to do this

The best way to initially approach this is to approach it as you would any novel software problem: do not reinvent the wheel and never build anything yourself that you could fairly appropriate from somebody else. There are many free datasets for a wide range of problems publicly available. Observe what types of data others who are solving problems similar to you have collected, and what you can learn about the datasets they used. It may be appropriate in the first instance, if a suitable dataset exists, to initially use a public dataset and iterate. If you do use other datasets, do make sure you respect the licenses that may come with them.

In most business cases, you will at some point end up collecting your own data. Even if you don't, it's important to be aware of what kind of data is desirable for AI and machine learning, and what kind of data is not. When looking at potential data, some key criteria to consider are:

- Accuracy
    - Does the data accurately measure a quantity you are interested in?
    - This sounds obvious, but not all data can be trusted. Data from asking human
    - participants questions, for example, can be inaccurate and contradictory.
- Completeness
    - Does the dataset represent a complete view of all data points of interest?

    - Does it have more data about some quantities than others? Should it?
    - Your models cannot learn from examples that are not in the data
- Relevance
    - To what extent is the collected data relevant to the measure of interest?
    - Including data that is only weakly relevant may cause more problems than it solves
- Missingness
    - Are there missing values in the data?
    - Distinct from completeness. Completeness is about overall coverage, missingness is about which bits of your collected data are not present.
- Timeliness
    - Is the data still relevant now?
- Subjectivity
    - AI methods are fundamentally quantitative, and deal best with quantitative data
- Attainability
    - Can the data be realistically obtained (and in the quantities required)?
- Standardization
    - Is the data collectable/attainable in a standardized format amenable to computation

What data you intend to collect is likely to be very tightly tied to where you collect your data. The best source of data is usually the source that gives the best data by the criteria we list above. This is not always the only consideration though, it is also wise to consider:

- Licensing. This applies both if you're using an existing dataset licensed by a third party (even a free one), or if your data might contain licensed work. As an example of the latter building a dataset of artwork may require you to consider the licenses of those artworks.
- Personal Data: classes of data (e.g., personal data) must be treated specially. More on this at the bottom of this section
- Special Cases: Depending on the data and end goal, you may be required to take additional steps in data collection. For example, data collected by experimenting on animals is likely to require extra licenses and oversight.

Implementing the Plan

We discussed supervised and unsupervised learning in the Define Goals and Metrics section. If you are dealing with a supervised learning problem (as is likely), the largest concern of data collection is how the data can be labeled. In a supervised learning application, we want to learn to predict some quantity from our data. To do that, we need examples which match our data and that quantity together. For example if we want an AI application that detects spam, we need to collect as data a set of emails, and divide them up into two categories - spam or not.

Fundamentally, you have two choices of how to do this. Firstly, you can contrive a way to achieve this automatically. If you are predicting how sales from your website occur based on how people engage with it, it might be a fairly simple affair for you to match these two bits of information up. Otherwise, if you can't contrive a way to do otherwise, the data must be labeled manually, by hand. For example, in a dataset of pictures of animals, the only way to effectively know what animal is present in the picture is to get a human to decide. In general, we wish to avoid this - for the purposes of this type of task, humans are expensive, prone to error and hard to scale.

Other than this, the process of how you will collect your data is simply the practical realization of what you've set out in the previous steps. Your focus here is making the process as simple and replicable as possible. As a general rule, the more automated the process can be, the better. Automated collection processes scale better, and involving human factors in the collection process is usually an excellent way to introduce an extra set of errors. Automated data collection isn't possible in every application though. If you do have to have people involved in your data collection process, try and work as hard as you can to maximize the consistency of the process for them.

It's worth saying that regardless of the initial choices you make, it's likely you'll revisit this step as you follow the other instructions and find out what works for you and what doesn't. This is perfectly fine, and it's much better initially to pick a dataset, make a start, and iterate, rather than trying to get it perfect the first time.

The criteria to complete this step are:

To create and implement (with exceptions, see below) a Data Collection Plan. This is a plan that details:

- What data you are going to collect
- Where/who you are going to collect it from
- How you are going to collect the data
- Any extra considerations

What data you are going to collect should include a data blueprint, a sample of exactly what you think the data you are going to collect should look like. The where/who should include specific populations you can feasibly target. The How should be a plan of action to collect the data from the first step from the groups you defined in the second step including, where required, a plan for how the data is to be labeled.

If your AI application is in the high AI Risk category, or you are dealing with personal data, some special considerations exist. Create this plan, but do not implement it until you have worked through the following:

For high AI Risk applications:

- You will be required to provide adequate documentation around data collection, and log all data collection activity. You will also be required to have human oversight over the data collection process. Make sure you visit the Documentation and Logging section of the guide that covers these areas before collecting any data
- You have a duty to make sure the data you collect is of a high quality to minimize discriminatory outcomes. Make sure you visit the Data Exploration and Biases section of the guide before collecting data.

For AI applications dealing with personal data:

- You must collect data according to GDPR regulation. This topic is expanded on in our Appendix on GDPR.

Examples

In this example, we work through the case of an oncologist looking to create an AI application to help other physicians detect the presence of tumors in a chest x-ray. There is an *example data set creation document<dataset_creation>*, and a description below:

The data I, as our imaginary oncologist, will need for my application is a set of chest x-rays, and whether they contain cancer or not.

Data	classification
x-ray0.png	cancerous
x-ray1.png	non-cancerous
x-ray2.png	cancerous
…	…

In respect to where I can find the data, a starting point is obviously the data that I can collect from my own patients. I may be able to get data from other patients from people in my professional network, or simply search online (there are several publicly available datasets on this particular topic).

In respect of how I can go about collecting (and labeling) my data. I can get chest X-rays from the sources described above. To label them, I can use my own expert knowledge, and/or ask other physicians to also contribute to corroborate.

In respect of extra considerations: I am working with personal health data. I'd likely need to obtain consent from the patients, must respect GDP, and may have additional requirements to fulfill in respect of my medical license, or an ethics board to satisfy. It's likely that this application would also fall into a high AI Risk category, and be subject to extra requirements.

Version Control, CI/CD for Data

Any electronic systems engineer should be familiar with version control. These ideas are just as important in developing AI applications as any other software product. In this step we explore:

- Version control systems for all data collection code
- Version control systems for all data collected

Our reasons for developing version control for data collection code are the same as they would be for any other software project. We have similar requirements of the datasets we collect with this code. Just like our code, our data is not something we can consider static. Not only is it possible we will collect more, but our existing data may be reorganized, fixed, or updated.

Version control for code is very well established, with a range of standard free tools (e.g. Git, Mercurial, Subversion) available. Version control of data requires a little more work. The standard tools used for code control are only appropriate for a (relatively) small number of small files, tracking a relatively small number of changes. Many datasets will not meet these criteria. In these cases we can either extend existing version control with “large file” control, or with an entirely separate data version control system. Free and Open Source examples of the above are Git LFS and Data Version Control respectively.

To complete this section, you must:

- Set up a version control system for your code
- Set up a version control system for your data
- Write a document about how they are to be used

Examples

Documentation

As with version control, good documentation is just as important, or perhaps even more so, as any other software development project. In this step we will explore:

- Documentation for the code and;
- Documentation for data

Compared to many software engineering projects, AI projects can often suffer from large variances in behavior due to the stochastic nature of the algorithms involved. A consequence of this is that it is imperative to maintain clear documentation. We must be able to clearly distinguish between acceptable variances in behavior due to irreducible randomness in our processes, and unacceptable variances in behavior due to mistakes.

Documentation should follow standard best practices. This is quite a large topic that has been extensively covered elsewhere (e.g. Write The Docs), that we won’t repeat in this guide. Instead, we devote this section to discussing the additional considerations required for documentation of data. Documentation for data serves not just a similar function to documentation for code in terms of bringing clarity and transparency to what has been done, but it also has a strong role in ensuring reproducibility. In many cases, and especially when dealing with challenging data (such as data involving human factors), how you went about collecting this data is just as important as what you ultimately collected. Ultimately, we suggest that documentation for data should consider the following three things:

What you collected. A good description should (if reasonable) include a way of positively identifying what was collected, where it was collected from and when.

How you collected it. A good description should both include a succinct high level description of what was done, and include enough detail to allow a full replication. Especially for complicated data collection paradigms, the devil is often in the details, and seemingly unimportant details can become important later.

Why you did it this way. Should both rationalize the process you took and, crucially, why you did this instead of other things. This gives important contextual hints to anyone trying to replicate your results that may be absent from a pure “how” description, and can help them avoid any problems you encountered in the process.

To achieve this we suggest that:

- Each piece of data comes with metadata, describing what it is
- Each group of data should be accompanied by a short document describing how and why it was collected.

This may sound like a significant overhead but, especially if your data is being collected digitally, the burden is not especially high. Populating metadata can often be significantly automated, and written documentation may overlap significantly with the existence and documentation of relevant code. For example, consider a dataset of images collected by a web scraper. It would be very easy to include a hash to positively identify what was collected, the web location it was collected from, and at what time during the scraping process. In respect of how this data was collected, the code for the webscraper itself provides a strong description of this. Even in respect of why it was done this way, the documentation for this code provides a significant amount of context.

High AI Risk applications:

- For high AI risk applications, documentation is no longer an internally driven process to improve productivity, but a (likely mandated) part of the requirement to demonstrate traceability and auditability of the software
- High risk AI Applications must have human oversight. Automated documentation of data collection will require a level of human oversight and validation.

The criteria to complete this step are:

- Creating documentation for all code written to this point, and standards for future code
- Creating documentation for any data collected to this point, and a process for documenting future data

Examples

Logging

Logging is a core requirement of the software development process. It must be accepted that software will break or not fulfill its function correctly, and when it does we need to be able to diagnose those faults effectively. Furthermore, software is rarely static, and in order to change it we must understand how it works. Finally, in many cases we may wish (or be required) to audit and review our software, and logging is an important part of this. In this step, we will explore:

- How to create a logging process for out data collection

When creating a logging process, the first question is always “What should we log?”. The best place to start with this is to instead start with the question “what questions do we want to answer about our software?”. Obviously, the answer to this will depend on the specifics of our data collection, but some recurring questions you will often need to answer are:

- Where did I get this piece of data?
- When did I get this piece of data?
- What was the state of my collection program when I collected this data?
- Was collecting this piece of data successful?
- Why was collecting this piece of data unsuccessful?

For high AI Risk applications:

- For high AI risk applications, logging is no longer optional. Logging of all actions relevant to proving compliance with the EU AI Act (see appendix) must be undertaken.

The criteria to complete this step are:

- Creating a logging process for all data collection code

Data Exploration

This step is separated from the Data Cleaning step for clarity, but in reality these two steps are likely to be quite closely linked together. In this step we will look at the process of exploring the data we collect.

We discussed previously in the guide that you may wish to return to earlier steps, and this data exploration step is one of the steps which is likely to encourage this.

In our steps so far we have designed an AI application, designed a dataset we think achieve our goals, and have taken the initial steps to ensure that there is a robust coding framework around this. Before going any further, we need to take a look at the data we have collected and try and understand it’s key characteristics, strengths, and weaknesses of the data to establish:

- An understanding of the the key characteristics, strengths, and weaknesses of the data
- What patterns and relationships exist in the data
- Whether it is likely to be useful for the purpose we intended
- What further data collection should fix, and what it should do more of

While this initial examination obviously can't understand the end result of our full AI pipeline ahead of time, we can build up an understanding of our data. It's likely that, especially for the first time, our data may not be exactly as we had hoped it would be.

Our data exploration process is about trying to digest information about our dataset. The methods we use to do this are very fundamental, intuitive ideas:

- Looking directly at the data and subsets thereof
- Trying to understand the data in an intuitive visual way
- Trying to understand the data through summaries and heuristics

The best approach for this will vary, but standard approaches for these are: * Tabular reports * Data Visualization * Data Profiling

Tabular reports. Forming tabular reports is a very simple way to look at our data directly. This is simply structuring our data set in a row/column format. There are no hard rules about how we might want to do this. We could look at subsets of the data, look at ordered columns, or anything else. Just looking at the raw data can often be very useful. We can check our intuitions about the data, identify potential patterns, and notice errors that may be hard to identify other ways.

Data Visualization. Directly examining data is useful in a way that should not be discarded. However, most of us will find it more useful to process data visually. There are a very large number of ways to do this. Edward Tuft lists a series of key ideas that are often cited for this:

- show the data
- induce the viewer to think about the substance rather than about methodology, graphic design, the technology of graphic production, or something else
- avoid distorting what the data has to say
- present many numbers in a small space
- make large data sets coherent
- encourage the eye to compare different pieces of data
- reveal the data at several levels of detail, from a broad overview to the fine structure
- serve a reasonably clear purpose: description, exploration, tabulation, or decoration
- be closely integrated with the statistical and verbal descriptions of a data set.

Good visualizations are often as much of an art as a science. As with many things for which this is the case, the best initial approaches are as follows:

- Start from the basics, the simple tools that everyone else's uses (line plot, scatter graphs, heatmaps)
- Unashamedly appropriate good idea from other people doing similar things
- Explore your own ideas to find out works for you and what doesn't

Data Profiling. Data profiling tries to capture yet another approach to understanding our data, this time through the use of summarizations and statistics about our data.

- What groupings (or clusters) within in our data
- Averages (mean, median, etc.)
- Spreads (standard deviation, quartiles, etc.)

As part of this we might want to explore how different bits of our data relate to each other. For example:

- Comparing different data
- Correlating different data

Alongside these broad techniques, we might also choose to do a detailed “drilldown” into our data and run a more detailed analysis of some parts. We might look at some more advanced statistics or visualizations of these subsets, or even perform a small scale trial run of some AI approaches we are considering later.

To criteria to complete this step is to:

- Create a data exploration process
- Create a data exploration report

Cleaning your Data

Validation and Testing

Scaling and Automation: Data Collection

Scaling and Automation: Data Storage

Scaling and Automation: Data Access

Training Your AI Application

Choosing Your AI approach

Version Control, CI/CD, Training

Documentation and Logging, Training

Data Pre-processing

Training Infrastructure

Creating a Training Process

Testing and Validation, Training

Exploring Outcomes and Biases

Scaling and Automation: Training Pipeline

Deploying your AI Application

Version Control, CI/CD, Deployment

Documentation and Logging, Deployment

Preparing a Trained Model

Deployment Infrastructure

Deploying Your Model

Testing and Validation, Deployment

Model Monitoring

Scaling and Automation: Deployment Pipeline

Examples

Example Engineering Document

Example Dataset Creation Document

Contributors

We thank all contributors and reviewers of this project.

Authors and Contributors

Alex Wang

Allison Lowndes

Andrew Rogoyski

Andy Bond

Arezou Nayebi

Charles Sturman

Dimitra Georgiadou

Elliot Stein

Gopal Ramchurn

James Watson

Jeremy Bennett

Lauren Thompson

Mark Zwolinski

Mike Bartley

Prethveraj M S

Tim Santos

William Jones

Reviewers

Indices and tables

- **genindex**
- **modindex**
- **search**